



## **Digital IC Design & Verification**

### **Lab Manual # 18 – RISC-V Datapath Implementation (Upper Immediate and Branches)**

**Release: 1.0 Date:**

**13-June-2024**

**NUST Chip Design Centre (NCDC), Islamabad, Pakistan**



**Copyrights** ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

## Revision History

| Revision Number | Revision Date | Revision By               | Nature of Revision | Approved By |
|-----------------|---------------|---------------------------|--------------------|-------------|
| 1.0             | 13/06/2024    | Ali Aqdas                 | Complete manual    | -           |
| 2.0             | 19/09/2025    | Umer Farooq / Fawad Ahmed | Complete Manual    |             |



## Contents

|                               |   |
|-------------------------------|---|
| Contents .....                | 2 |
| Objective.....                | 3 |
| Tools .....                   | 3 |
| Instructions.....             | 3 |
| RISC-V Datapath .....         | 4 |
| Branches.....                 | 4 |
| Task 1 .....                  | 5 |
| Upper Immediate (U-Type)..... | 5 |
| Task 2 .....                  | 6 |



## Objective

The objective of this lab is to

- Extend the previously designed datapath to include branches and upper immediates

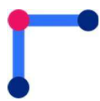
## Tools

- Cadence Xcelium
- Xilinx Vivado

## Instructions

The students are required to design and submit the updated files for evaluation of this lab. An additional file for branch comparison has to be added to the submission.

|                   |        |                                                                                                                                                             |
|-------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ./<student-name>/ | ./rtl/ | program_counter.sv<br>inst_mem.sv<br>data_mem.sv<br>reg_file.sv<br>imm_gen.sv<br>main_ctrl.sv<br>alu_ctrl.sv<br>alu.sv<br>top.sv                            |
|                   | ./tb/  | program_counter_tb.sv<br>inst_mem_tb.sv<br>data_mem_tb.sv<br>reg_file_tb.sv<br>imm_gen_tb.sv<br>main_ctrl_tb.sv<br>alu_ctrl_tb.sv<br>alu_tb.sv<br>top_tb.sv |



## RISC-V Datapath

The datapath designed so far that includes all the **R-Type**, **J-Type**, **S-Type**, and **I-Type Instructions**. The next goal is to extend the design to include upper immediate (U-Type) and Branches (B-Type).

### Branches

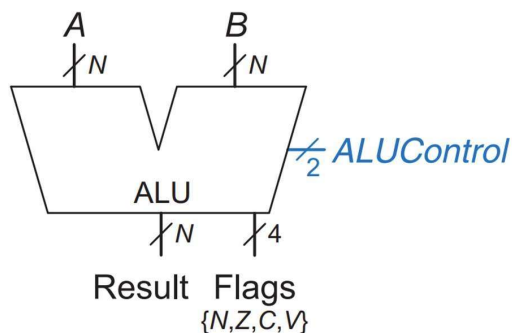
There are several types of branch instructions having B-Type Encoding and listed in the table below



| op           | funct3 | funct7 | Type | Instruction          | Description          | Operation                |
|--------------|--------|--------|------|----------------------|----------------------|--------------------------|
| 1100011 (99) | 000    | -      | B    | beq rs1, rs2, label  | branch if =          | if (rs1 == rs2) PC = BTA |
| 1100011 (99) | 001    | -      | B    | bne rs1, rs2, label  | branch if ≠          | if (rs1 ≠ rs2) PC = BTA  |
| 1100011 (99) | 100    | -      | B    | blt rs1, rs2, label  | branch if <          | if (rs1 < rs2) PC = BTA  |
| 1100011 (99) | 101    | -      | B    | bge rs1, rs2, label  | branch if ≥          | if (rs1 ≥ rs2) PC = BTA  |
| 1100011 (99) | 110    | -      | B    | bltu rs1, rs2, label | branch if < unsigned | if (rs1 < rs2) PC = BTA  |
| 1100011 (99) | 111    | -      | B    | bgeu rs1, rs2, label | branch if ≥ unsigned | if (rs1 ≥ rs2) PC = BTA  |

When the branch condition becomes true the processor jumps to the label listed in the instruction. Essentially, label is encoded as an immediate value that specifies the offset from current value of program counter. i.e. **PC = PC + Offset (Label)**.

It might seem evident, but for sake of clarity we explain that arithmetic logic unit (ALU) is busy computing program counter address, and we need to **we will use ALU for comparing the branch conditions**.



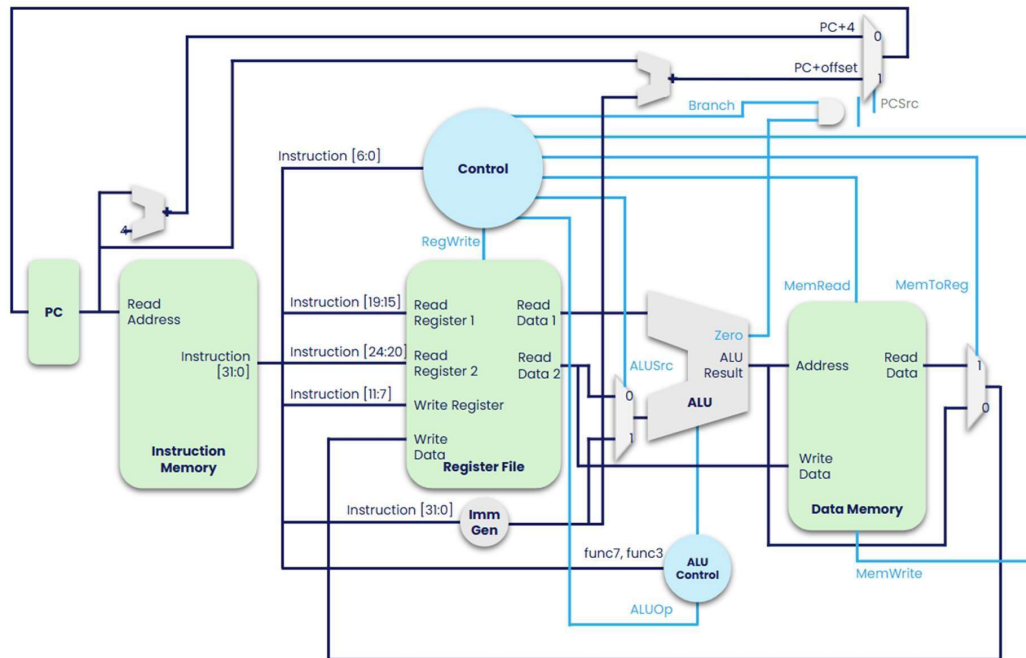
Here are the conditions of ALU for executing 6 branch instructions.

- If  $RS1 - RS2 = 0$ , this means that both values are equal, so we will set Zero Flag Signal = 1 for beq otherwise Zero Flag Signal = 0 for bne
- If  $RS1 - RS2 = -ve$ , this means that  $RS1 < RS2$ , so we will set Negative Flag Signal = 1 for blt otherwise Negative Flag Signal = 0 for bgt. Similarly same condition for bltu & bgtu



## Task 1

Extend the design of your datapath according to this diagram. After extending your design, it may look along the lines of the following figure



## Upper Immediate (U-Type)

The next goal is to extend the design to include upper immediate instructions, encoded as U- Type and listed in the table below.

| imm <sub>31:12</sub> |        |        |      | rd              | op                        | U-Type                   |
|----------------------|--------|--------|------|-----------------|---------------------------|--------------------------|
| op                   | funct3 | funct7 | Type | Instruction     | Description               | Operation                |
| 0110111 (55)         | -      | -      | U    | lui rd, upimm   | load upper immediate      | rd = {upimm, 12'b0}      |
| 0010111 (23)         | -      | -      | U    | auipc rd, upimm | add upper immediate to PC | rd = {upimm, 12'b0} + PC |



## Task 2

Extend your datapath to include the upper immediate type instructions. Now that your datapath is complete, it may look along the lines of following:

